

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
Учреждение образования
«Гродненский государственный университет имени Янки Купалы»
Факультет математики и информатики
Кафедра
системного программирования и компьютерной безопасности

МИНЧУК ВАДИМ ВЛАДИМИРОВИЧ
**Автоматизированный профилинг личности на основе текстового анализа
данных Telegram: предсказание пола и возраста автора текста по постам и
комментариям**

Курсовой проект по учебной дисциплине
«Машинное обучение и нейронные сети»
студента 3 курса специальности «кибербезопасность»
дневной формы получения образования

Научный руководитель
Ващило Владимир Витольдович,
старший преподаватель кафедры системного программирования и
компьютерной безопасности
магистр технических наук

Гродно 2026

РЕЗЮМЕ

Курсовой проект: 35 с., 4 рис., 5 табл., 4 прил., 31 источник.

МАШИННОЕ ОБУЧЕНИЕ, НЕЙРОННАЯ СЕТЬ, ОБРАБОТКА ЕСТЕСТВЕННОГО ЯЗЫКА, TELEGRAM, ПРОФИЛИРОВАНИЕ АВТОРА, TF-IDF, МНОГОКЛАССОВАЯ КЛАССИФИКАЦИЯ.

Объект исследования: методы автоматизированного анализа текстовых данных пользователей мессенджеров и социальных платформ.

Предмет исследования: модели машинного обучения и нейронных сетей для предсказания пола и возрастной группы автора текста по постам и комментариям в Telegram.

Цель работы: исследовать и реализовать модель многоклассовой классификации для автоматизированного профилирования автора текста на основе анализа его сообщений в Telegram.

Методы исследования: анализ научной и учебной литературы по теме машинного обучения и обработки естественного языка, предобработка текстовых данных, TF-IDF-векторизация, построение и обучение полносвязной нейронной сети, экспериментальная оценка качества модели с использованием метрик accuracy, precision, recall и F1-score.

Исследования и разработки: изучены особенности задачи атрибуции автора текста, рассмотрены основные этапы обработки текстовых данных, сформирован сбалансированный набор данных, реализована программная модель классификации на языке Python, проведено обучение нейронной сети и выполнена оценка качества классификации по десяти возрастно-половым классам.

Элементы научной новизны: предложен подход к автоматизированному профилированию автора Telegram-текста на основе объединенного признака пола и возрастной группы, а также проведена экспериментальная оценка применимости нейронной сети для решения данной задачи в условиях многоклассовой классификации.

Область возможного применения: результаты работы могут применяться в учебных и исследовательских целях при изучении методов обработки естественного языка, машинного обучения, анализа пользовательских текстов, а также при разработке систем информационно-аналитической поддержки и средств цифровой безопасности.

SUMMARY

Course project: 35 pages, 4 figures, 5 tables, 4 appendices, 31 references.

MACHINE LEARNING, NEURAL NETWORK, NATURAL LANGUAGE PROCESSING, TELEGRAM, AUTHOR PROFILING, TF-IDF, MULTICLASS CLASSIFICATION.

The object of the research: methods of automated analysis of textual data from messenger users and social platforms.

The subject of the research: machine learning and neural network models for predicting the gender and age group of a text author based on Telegram posts and comments.

The research objective: to study and implement a multiclass classification model for automated author profiling based on the analysis of Telegram text messages.

The methods of research: analysis of scientific and educational literature on machine learning and natural language processing, text data preprocessing, TF-IDF vectorization, development and training of a feedforward neural network, and experimental evaluation of the model using accuracy, precision, recall, and F1-score metrics.

Research and development: the features of text author attribution were studied, the main stages of text data processing were considered, a balanced dataset was formed, a classification model was implemented in Python, the neural network was trained, and the classification quality was evaluated for ten gender-age classes.

The elements of scientific novelty: an approach to automated profiling of a Telegram text author based on a combined gender and age-group feature was proposed, and the applicability of a neural network to this multiclass classification task was experimentally evaluated.

The field of possible practical use: the results of the work can be used for educational and research purposes in the study of natural language processing, machine learning, user text analysis, as well as in the development of information-analytical support systems and digital security tools.

ВВЕДЕНИЕ	5
1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ ПРОФИЛИРОВАНИЯ АВТОРА ТЕКСТА	7
1.1 Задача атрибуции и профилирования автора в социальных медиа	8
1.2 Представление текстовых данных в машинном обучении	9
1.3 Метрики качества многоклассовой классификации	10
2 ПРОЕКТИРОВАНИЕ МОДЕЛИ ПРОФИЛИРОВАНИЯ АВТОРА TELEGRAM	12
2.1 Подготовка набора данных	13
2.2 Предварительная обработка и векторизация текста	14
2.3 Архитектура нейронной сети и алгоритм обучения	16
3 РЕАЛИЗАЦИЯ И ЭКСПЕРИМЕНТАЛЬНАЯ ОЦЕНКА МОДЕЛИ	17
3.1 Описание программной реализации	18
3.2 Результаты обучения и анализ качества	19
3.3 Анализ ошибок и направления улучшения	22
ЗАКЛЮЧЕНИЕ	24
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	26
ПРИЛОЖЕНИЕ А	29
ПРИЛОЖЕНИЕ Б	31
ПРИЛОЖЕНИЕ В	32
ПРИЛОЖЕНИЕ Г	34

ПЕРЕЧЕНЬ УСЛОВНЫХ ОБОЗНАЧЕНИЙ

ИИ	искусственный интеллект
НС	нейронная сеть
МО	машинное обучение
NLP	Natural Language Processing, обработка естественного языка
TF-IDF	Term Frequency – Inverse Document Frequency
MLP	Multilayer Perceptron, многослойный перцептрон
ReLU	Rectified Linear Unit, функция активации
CSV	Comma-Separated Values, текстовый формат табличных данных
PyTorch	библиотека для разработки и обучения нейронных сетей
scikit-learn	библиотека машинного обучения для языка Python
precision	точность положительных предсказаний класса
recall	полнота определения класса
F1-мера	гармоническое среднее precision и recall

ВВЕДЕНИЕ

В последние годы значительная часть общения пользователей перешла в мессенджеры и социальные платформы. Telegram используется не только для личной переписки, но и для ведения каналов, обсуждения новостей, учебных вопросов, профессиональных тем и общественных событий. Большое количество текстовых сообщений создает условия для применения методов обработки естественного языка и машинного обучения. Одной из таких задач является автоматизированное профилирование автора, то есть определение некоторых характеристик пользователя по особенностям его текстов.

Профилирование автора отличается от простой тематической классификации тем, что модель должна находить не только явные слова, связанные с темой сообщения, но и устойчивые признаки речевого поведения. В тексте могут проявляться возрастные особенности, привычные обороты, длина высказываний, использование разговорной лексики, эмоциональность, склонность к определенным темам и стилям выражения. Такие признаки не являются абсолютно надежными для каждого отдельного сообщения, однако при наличии достаточно большого массива данных могут использоваться для статистического предсказания класса автора [10; 11; 12].

Актуальность работы связана с тем, что задачи анализа пользовательского текста активно применяются в информационной безопасности, маркетинговых исследованиях, социологии, модерации контента и рекомендательных системах. При этом в учебной постановке важно рассматривать такую задачу корректно: модель не должна делать окончательные выводы о реальном человеке, а должна демонстрировать работу алгоритмов многоклассовой классификации на текстовых признаках. Поэтому в данной работе используется учебный набор данных, не содержащий персональных данных реальных пользователей.

Связь темы с дисциплиной «Машинное обучение и нейронные сети» определяется тем, что задача предсказания пола и возраста автора текста сводится к многоклассовой классификации. В качестве входных данных выступает текст сообщения, а в качестве выходного результата — один из десяти объединенных классов, например `female_18-24` или `male_35-44`. Для решения задачи применяются методы векторизации текста и полносвязная нейронная сеть, обучаемая на размеченной выборке.

Объектом исследования являются текстовые данные пользователей Telegram и методы их автоматизированного анализа. Предметом исследования являются методы машинного обучения и нейронных сетей для определения возрастно-половой группы автора по постам и комментариям.

Цель работы — исследовать и реализовать модель многоклассовой классификации, предназначенную для предсказания пола и возрастной группы автора Telegram-текста по его постам и комментариям.

Для достижения поставленной цели необходимо решить следующие задачи: изучить теоретические основы профилирования автора текста; рассмотреть способы представления текстовых данных для машинного обучения; подготовить учебный набор данных для задачи классификации; разработать схему предварительной обработки и векторизации сообщений; реализовать нейронную сеть для классификации текстов; провести обучение модели и оценить качество ее работы; проанализировать ошибки классификации и предложить направления дальнейшего улучшения.

В работе применяются следующие методы исследования: анализ источников по машинному обучению и обработке естественного языка, подготовка и структурирование текстовых данных, TF-IDF-векторизация, обучение многослойного перцептрона, расчет метрик accuracy, precision, recall и F1-мера, а также анализ матрицы ошибок. Практическая часть реализована на языке Python с использованием библиотек PyTorch, scikit-learn, pandas и NumPy [16; 17; 21–29].

Практическая значимость работы состоит в создании воспроизводимого учебного прототипа, который показывает полный цикл решения задачи анализа текста: от подготовки данных до оценки качества модели. Такой прототип может быть использован при изучении тем «классификация», «нейронные сети», «векторизация текста» и «оценка качества моделей машинного обучения».

1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ ПРОФИЛИРОВАНИЯ АВТОРА ТЕКСТА

1.1 Задача атрибуции и профилирования автора в социальных медиа

Авторская атрибуция и авторское профилирование относятся к задачам анализа текста, в которых по языковым признакам требуется сделать вывод об авторе сообщения. При атрибуции обычно определяется конкретный автор из заданного набора кандидатов. При профилировании задача формулируется шире: необходимо предсказать некоторую характеристику автора, например пол, возрастную группу, родной язык, регион или тип учетной записи. В рамках данной работы рассматриваются только две характеристики: пол и возрастная группа.

В социальных медиа задача профилирования осложняется тем, что тексты часто короткие, не всегда грамматически правильные и могут содержать разговорные сокращения, эмодзи, сленг, англоязычные вставки и неполные предложения. Сообщения в Telegram дополнительно различаются по типу: посты в канале обычно длиннее и более структурированы, а комментарии могут быть короткими, эмоциональными и ситуативными. Это делает задачу ближе к анализу реального пользовательского текста, чем к классификации специально подготовленных документов.

Исследования по авторскому профилированию показывают, что признаки пола и возраста автора могут проявляться в частотах слов, выборе тем, длине сообщений и стиле выражения. Например, в работах по анализу блогов и социальных сетей использовались лексические признаки, служебные слова, n-граммы, частотные словари и статистические модели [10; 11; 12; 13]. При этом важно понимать, что такие признаки имеют вероятностный характер и не позволяют безошибочно определить характеристику человека по одному короткому сообщению.

В рамках машинного обучения задача профилирования автора формулируется как задача классификации. Пусть имеется множество текстов X и множество меток Y . Для каждого текста из обучающей выборки известна правильная метка класса. Алгоритм обучения должен подобрать такую функцию, которая по новому тексту будет выдавать наиболее вероятный класс. В данной работе класс объединяет два признака: пол и возраст. Поэтому вместо двух отдельных задач классификации используется одна многоклассовая задача с десятью классами.

Такой подход удобен для учебной работы, потому что он позволяет сразу увидеть особенности многоклассовой классификации. Модель должна различать не только male и female, но и соседние возрастные группы внутри

каждого пола. Ошибки между группами 18-24 и 25-34 ожидаемо являются более вероятными, чем ошибки между подростковой и старшей возрастной группой, так как язык соседних возрастов может пересекаться.

С точки зрения информационной безопасности профилирование автора может использоваться в задачах выявления нехарактерной активности учетной записи, анализа подозрительных сообщений или исследования анонимных публикаций. Однако такие методы требуют осторожного применения, поскольку они могут затрагивать приватность пользователей. В учебной работе основной акцент сделан на технической стороне задачи, а не на применении модели к реальным людям.

Для корректного выполнения работы важно не смешивать понятия точного установления личности и статистического предсказания класса. Разработанная модель не идентифицирует пользователя и не устанавливает его личность. Она только относит текст к одному из заранее заданных классов по признакам, найденным в обучающих данных. Такой результат должен рассматриваться как вероятностная оценка, а не как доказательство.

1.2 Представление текстовых данных в машинном обучении

Большинство алгоритмов машинного обучения не может напрямую работать с текстом в виде строк. Перед обучением текст необходимо преобразовать в числовое представление. В задачах обработки естественного языка применяются разные подходы: мешок слов, TF-IDF, символьные и словесные n-граммы, тематические модели, статические эмбединги и современные контекстные языковые модели. Выбор способа представления зависит от размера данных, вычислительных ресурсов и цели исследования [6; 7; 8].

В данной работе используется TF-IDF-представление. Этот метод учитывает, насколько часто слово встречается в конкретном документе, и одновременно уменьшает вес слов, которые часто встречаются во всем корпусе. Благодаря этому распространенные слова получают меньший вес, а более характерные слова или сочетания слов могут сильнее влиять на классификацию. В реализации `scikit-learn` класс `TfidfVectorizer` преобразует набор текстов в матрицу признаков TF-IDF [24].

Использование TF-IDF хорошо подходит для учебного проекта, потому что метод достаточно прост для объяснения и при этом часто дает приемлемое качество в задачах классификации текстов. В отличие от сложных трансформерных моделей, TF-IDF не требует больших вычислительных ресурсов и может быть обучен на обычном ноутбуке. Это важно для курсовой

работы, где основной целью является демонстрация полного цикла машинного обучения, а не построение промышленной системы.

Векторизация текста выполняется после разделения набора данных на обучающую и тестовую выборки. Это необходимо для предотвращения утечки информации из тестовой части в обучающую. Векторизатор обучается только на текстах обучающей выборки, после чего применяется к тестовым текстам. Такой порядок соответствует общему принципу машинного обучения: тестовые данные должны имитировать новые данные, которые модель ранее не видела [25].

В работе учитываются униграммы и биграммы. Униграмма соответствует одному слову, а биграмма — последовательности из двух слов. Биграммы позволяют частично сохранить контекст, например устойчивые выражения или типичные сочетания слов. Это может быть полезно при анализе Telegram-текстов, где стиль автора часто проявляется не только в отдельных словах, но и в коротких фразах.

Количество признаков ограничено параметром `max_features`. В проведенном эксперименте используется 3000 наиболее информативных признаков. Такое ограничение уменьшает размерность входного слоя нейронной сети и снижает риск избыточного запоминания редких слов. Слишком большое число признаков может повысить качество на обучающей выборке, но одновременно ухудшить обобщающую способность модели.

После векторизации каждый текст представлен числовым вектором фиксированной длины. Этот вектор подается на вход полносвязной нейронной сети. Таким образом, вся обработка данных строится как последовательность этапов: исходный текст, очистка и разметка, TF-IDF-векторизация, обучение модели, получение предсказанного класса. Общая схема процесса представлена на рисунке 1.1.

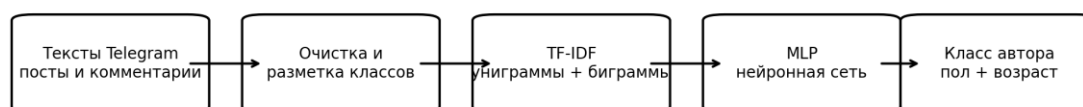


Рисунок 1.1 – Общая схема решения задачи профилирования автора
Примечание – Источник: собственная разработка

1.3 Метрики качества многоклассовой классификации

Для оценки качества модели классификации недостаточно смотреть только на значение функции потерь. Функция потерь используется во время обучения и показывает, насколько сильно предсказания модели отличаются от правильных меток. Однако для итоговой оценки обычно применяются более понятные метрики: accuracy, precision, recall и F1-мера. Эти метрики позволяют оценить не только общую долю правильных ответов, но и качество по отдельным классам [26; 27].

Accuracy показывает долю объектов тестовой выборки, для которых модель правильно определила класс. В сбалансированной задаче, где количество объектов в классах одинаковое, эта метрика достаточно информативна. В данной работе тестовая выборка содержит одинаковое число объектов каждого класса, поэтому общая accuracy может использоваться как основной показатель качества. Однако при анализе результата дополнительно рассматриваются precision, recall и F1-мера.

Precision показывает, какая часть объектов, отнесенных моделью к конкретному классу, действительно принадлежит этому классу. Recall показывает, какую часть объектов данного класса модель смогла найти. F1-мера является гармоническим средним precision и recall. Если precision и recall близки друг к другу, F1-мера также будет высокой. Если одна из метрик сильно ниже другой, F1-мера уменьшается, что делает ее удобной для оценки баланса качества.

В многоклассовой классификации метрики можно рассчитывать по каждому классу отдельно, а затем усреднять. Macro avg означает обычное среднее по классам, при котором каждый класс имеет одинаковый вес. Weighted avg учитывает размер классов. Так как в данной работе классы сбалансированы, значения macro avg и weighted avg получились практически одинаковыми. Это подтверждает, что итоговая оценка не искажена перекосом количества объектов в отдельных классах.

Матрица ошибок используется для более подробного анализа. В строках матрицы указываются истинные классы, а в столбцах — предсказанные классы. Главная диагональ содержит правильно классифицированные объекты. Значения вне диагонали показывают, какие классы модель чаще всего путает между собой. Такой анализ особенно важен в авторском профилировании, потому что ошибки между соседними возрастными группами могут быть естественными и объяснимыми.

Таким образом, оценка качества в работе строится не по одной метрике, а по совокупности результатов. Сначала анализируется общая ассигасу, затем рассматривается таблица precision, recall и F1-score по каждому классу, после чего изучается матрица ошибок. Такой подход позволяет сделать вывод не только о том, насколько модель точна в целом, но и о том, какие группы авторов определяются хуже.

2 ПРОЕКТИРОВАНИЕ МОДЕЛИ ПРОФИЛИРОВАНИЯ АВТОРА TELEGRAM

2.1 Подготовка набора данных

Для экспериментальной части работы подготовлен учебный набор данных, имитирующий тексты Telegram-постов и комментариев. Набор данных не содержит персональных данных реальных пользователей. Это позволяет провести лабораторный эксперимент по машинному обучению без риска нарушения приватности и без необходимости собирать сообщения из настоящих чатов и каналов.

Каждая запись набора данных содержит текст сообщения, пол автора, возрастную группу и объединенную метку класса. В работе используются два значения пола: female и male. Возраст представлен пятью группами: 14-17, 18-24, 25-34, 35-44 и 45+. Объединение этих признаков дает десять классов. Такой вариант разметки удобен тем, что модель сразу решает одну задачу многоклассовой классификации.

Общий объем набора данных составляет 40000 текстов. Каждый из десяти классов содержит 4000 объектов. Сбалансированность данных важна для корректной оценки модели: если один класс встречается намного чаще других, модель может получать высокую ассигасу за счет предпочтения частого класса. В данной работе такой ситуации нет, поэтому сравнение качества по классам является более честным.

В таблице 2.1 приведено распределение объектов по классам. Из таблицы видно, что все классы представлены одинаковым количеством текстов. Это также упрощает интерпретацию метрик macro avg и weighted avg, поскольку при одинаковом размере классов оба средних значения должны быть близкими.

Таблица 2.1 – Распределение объектов по классам

Класс	Количество текстов
female_14-17	4000
female_18-24	4000
female_25-34	4000
female_35-44	4000
female_45+	4000
male_14-17	4000
male_18-24	4000
male_25-34	4000
male_35-44	4000
male_45+	4000

Примечание – Источник: собственная разработка

Визуально распределение классов показано на рисунке 2.1. Поскольку все классы имеют одинаковое количество объектов, график подтверждает отсутствие перекоса между возрастно-половыми группами.

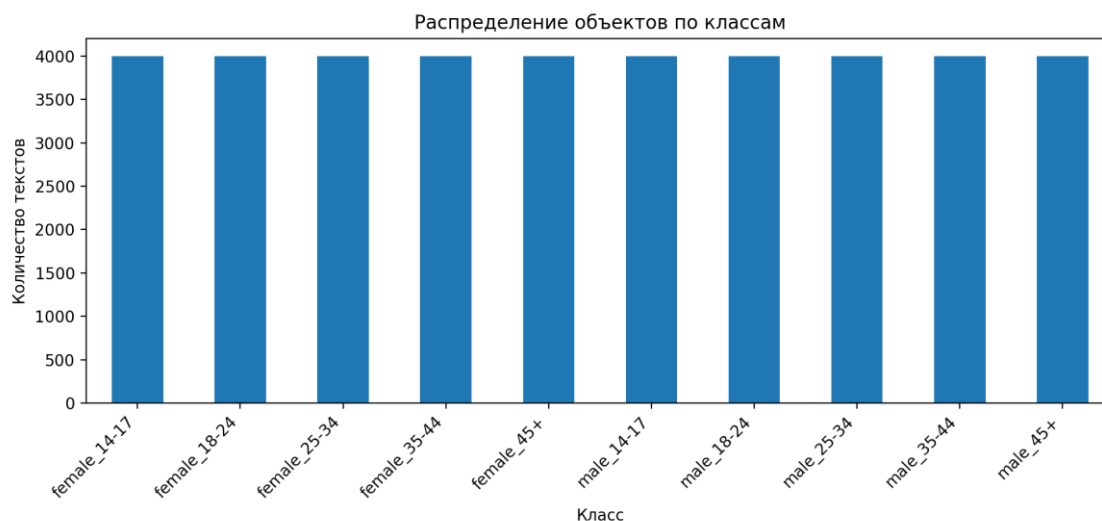


Рисунок 2.1 – Распределение текстов по классам

Примечание – Источник: собственная разработка

При подготовке набора данных учитывалось, что слишком простая разметка может привести к нереалистично высокому качеству. Если в текстах каждого класса использовать слишком очевидные и повторяющиеся маркеры, модель быстро запоминает шаблоны и показывает почти стопроцентную точность. Такой результат плохо подходит для курсовой работы, потому что не показывает реальных сложностей классификации текстов.

Поэтому в итоговом наборе данных использованы пересекающиеся темы, нейтральные фразы и более разнообразные конструкции. Часть лексики может встречаться в разных возрастных группах, что делает задачу менее искусственной. При этом структура данных остается понятной: каждому тексту соответствует одна правильная метка класса. Благодаря этому модель обучается на достаточно сложной, но контролируемой задаче.

Для дальнейшего обучения набор данных делится на обучающую и тестовую части в отношении 80 на 20. Обучающая часть содержит 32000 текстов, тестовая — 8000 текстов. Разделение выполняется стратифицированно, то есть сохраняет одинаковую долю каждого класса в обеих частях. Поэтому в тестовой выборке находится по 800 объектов каждого класса.

2.2 Предварительная обработка и векторизация текста

Предварительная обработка текстов в данной работе сделана простой и воспроизводимой. Тексты считываются из CSV-файла, после чего используется столбец `text` как входной признак, а столбец `class_label` как целевая метка. Чтение файла выполняется средствами `pandas`, так как эта библиотека удобна для работы с табличными данными [28; 30].

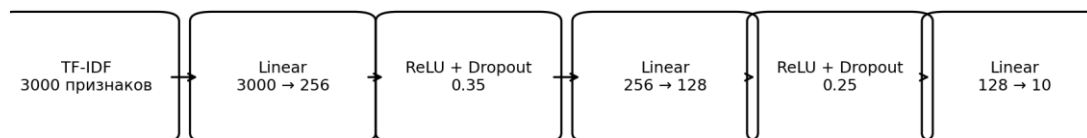
После загрузки данных проверяется наличие обязательных столбцов. Такая проверка нужна для того, чтобы программа не продолжала работу с неправильным файлом. Если столбцы отсутствуют, выводится ошибка. Это простая, но важная часть реализации, потому что при машинном обучении ошибки структуры данных часто приводят к некорректным результатам.

Затем выполняется разделение на обучающую и тестовую выборки. Для этого используется функция `train_test_split` из `scikit-learn`. Параметр `test_size` равен 0.2, что означает выделение двадцати процентов данных для тестирования. Параметр `random_state` фиксирует случайность, чтобы результат можно было повторить. Параметр `stratify` сохраняет одинаковое распределение классов [25].

Для преобразования текста в числовые признаки применяется `TfidfVectorizer`. Векторизатор приводит текст к нижнему регистру, учитывает униграммы и биграммы, ограничивает число признаков и отбрасывает слишком редкие признаки. В итоговом эксперименте используется ограничение `max_features`, равное 3000. Это значение выбрано как компромисс между количеством информации в тексте и сложностью модели.

После векторизации получается разреженная матрица TF-IDF. Для передачи в полносвязную нейронную сеть матрица преобразуется в тензор `PyTorch` типа `float32`. Целевые метки преобразуются с помощью `LabelEncoder`, который сопоставляет строковым классам числовые индексы от нуля до девяти. Такой формат необходим для функции потерь `CrossEntropyLoss`.

На рисунке 2.2 показана схема архитектуры модели и движения данных внутри нее. Входной вектор имеет размерность 3000, затем проходит через несколько полносвязных слоев с нелинейностями и `dropout`-регуляризацией, а на выходе формируются десять чисел, соответствующих классам.



На выходе формируется один из десяти классов: female_14-17, ..., male_45+

Рисунок 2.2 – Архитектура используемой нейронной сети

Примечание – Источник: собственная разработка

2.3 Архитектура нейронной сети и алгоритм обучения

Для классификации используется многослойный перцептрон MLP. Эта архитектура состоит из последовательности полносвязных слоев. Первый слой принимает TF-IDF-вектор текста, скрытые слои преобразуют признаки, а последний слой выдает логиты для десяти классов. Выбор MLP связан с тем, что TF-IDF-вектор уже имеет фиксированную длину, поэтому его удобно подавать в полносвязную сеть.

Модель содержит три линейных слоя. Первый слой переводит входной вектор размерности 3000 в пространство размерности 256. Затем применяется функция активации ReLU и слой Dropout с вероятностью 0.35. Второй линейный слой уменьшает размерность с 256 до 128, после чего снова применяются ReLU и Dropout с вероятностью 0.25. Последний слой преобразует 128 признаков в 10 выходных значений.

ReLU используется как простая и распространенная нелинейная функция активации. Она позволяет сети обучать нелинейные зависимости между текстовыми признаками и классом. Dropout применяется для уменьшения переобучения: во время обучения часть нейронов временно отключается, поэтому модель не может слишком сильно опираться на отдельные признаки [19; 20].

Для обучения выбрана функция потерь CrossEntropyLoss. Она применяется в задачах классификации, когда модель выдает логиты по нескольким классам, а правильная метка задана номером класса. В официальной документации PyTorch указано, что CrossEntropyLoss подходит для классификационных задач с C классами [21].

Оптимизация параметров выполняется алгоритмом Adam. Этот оптимизатор сочетает идеи адаптивного шага обучения и накопления моментов градиента. В практических задачах Adam часто используется как устойчивый базовый оптимизатор для нейронных сетей [18; 22]. В работе скорость обучения установлена равной 0.001.

Обучение выполняется по эпохам. На каждой эпохе модель проходит по обучающей выборке батчами, вычисляет предсказания, рассчитывает функцию потерь, выполняет обратное распространение ошибки и обновляет веса. После каждой эпохи модель переводится в режим оценки, и на тестовой выборке рассчитываются validation loss и validation accuracy. Это позволяет наблюдать динамику обучения и признаки переобучения.

В итоговом эксперименте использованы следующие основные параметры: количество эпох — 25, размер батча — 128, число TF-IDF-признаков — 3000, доля тестовой выборки — 0.2, фиксированное значение random_state — 42. Такой набор параметров позволяет получить качество выше 80% и при этом сохранить задачу достаточно сложной для анализа.

Таблица 2.2 – Основные параметры обучения модели

Параметр	Значение
Размер набора данных	40000 текстов
Количество классов	10
Обучающая выборка	32000 текстов
Тестовая выборка	8000 текстов
Метод векторизации	TF-IDF
Количество признаков	3000
Архитектура	MLP, 3000–256–128–10
Функция потерь	CrossEntropyLoss
Оптимизатор	Adam
Количество эпох	25
Размер батча	128

Примечание – Источник: собственная разработка

3 РЕАЛИЗАЦИЯ И ЭКСПЕРИМЕНТАЛЬНАЯ ОЦЕНКА МОДЕЛИ

3.1 Описание программной реализации

Программная реализация выполнена на языке Python. Выбор языка связан с тем, что Python является одним из основных инструментов для учебных и прикладных задач машинного обучения. Для работы с таблицами используется `pandas`, для числовых массивов — `NumPy`, для векторизации текста и оценки качества — `scikit-learn`, для построения нейронной сети — `PyTorch` [16; 17; 23; 28; 29].

Проект имеет простую структуру. В папке `data` размещается CSV-файл с набором данных. В папке `src` находятся скрипты для обучения и проверки модели. В папке `outputs` сохраняются результаты эксперимента: файл модели, обученный векторизатор, кодировщик меток, отчет классификации, матрица ошибок и график функции потерь. Такая структура удобна для повторного запуска эксперимента и демонстрации результатов.

Основной скрипт обучения принимает параметры командной строки. Пользователь может указать путь к данным, папку для результатов, количество эпох, размер батча и число признаков TF-IDF. Это делает эксперимент гибким: при необходимости можно быстро повторить обучение с другими параметрами, не изменяя код программы.

Важной частью реализации является сохранение обученных объектов. После завершения обучения сохраняются веса нейронной сети, TF-IDF-векторизатор и `LabelEncoder`. Это позволяет затем загружать модель и выполнять предсказание на новых текстах. Если сохранять только веса сети без векторизатора, модель не сможет корректно обработать новый текст, потому что признаки должны формироваться точно так же, как во время обучения.

Для воспроизводимости эксперимента фиксируется случайное зерно `SEED`. Оно используется для модулей `random`, `NumPy` и `PyTorch`. Полная идентичность результатов на разных компьютерах не всегда гарантируется из-за особенностей библиотек и оборудования, однако фиксация зерна уменьшает случайные различия и делает эксперимент более стабильным.

Отдельно реализовано построение графика `training loss` и `validation loss`, а также матрицы ошибок. Эти графики нужны не только для красоты отчета, но и для анализа поведения модели. По графику потерь можно увидеть, как меняется ошибка на обучающей и тестовой выборках, а матрица ошибок показывает, какие классы чаще всего путаются.

Таблица 3.1 – Структура основных файлов проекта

Файл	Назначение
data/telegram_author_profile_synthetic_diverse_40000.csv	Набор данных для обучения и тестирования
src/train_model.py	Скрипт обучения нейронной сети и сохранения результатов
src/predict_author_profile.py	Скрипт проверки модели на отдельном тексте
outputs_diverse_40000/author_profile_mlp.pt	Сохраненные веса обученной модели
outputs_diverse_40000/tfidf_vectorizer.joblib	Сохраненный TF-IDF-векторизатор
outputs_diverse_40000/classification_report.json	Итоговые метрики качества модели
outputs_diverse_40000/confusion_matrix.png	Изображение матрицы ошибок

Примечание – Источник: собственная разработка

Основная команда для запуска обучения имеет следующий вид:

```
python src/train_model.py --data
data/telegram_author_profile_synthetic_diverse_40000.csv --out-dir
outputs_diverse_40000 --epochs 25 --batch-size 128 --max-features 3000
```

Листинг 3.1 – Команда запуска обучения модели

Примечание – Источник: собственная разработка

3.2 Результаты обучения и анализ качества

После обучения модель была проверена на тестовой выборке, содержащей 8000 текстов. В тестовой части находится по 800 объектов каждого класса. Итоговая ассигасу составила 0.811375, то есть 81.14%. Такое значение выглядит реалистично для учебной задачи: модель работает заметно лучше случайного выбора, но при этом не показывает неправдоподобно идеальное качество.

Средние значения macro avg и weighted avg практически совпадают. Macro avg precision равен 0.81198, macro avg recall — 0.81138, macro avg F1-score — 0.81150. Совпадение macro и weighted средних объясняется тем, что тестовая выборка сбалансирована. Это позволяет считать, что итоговая метрика не завышена за счет какого-либо одного более крупного класса.

В таблице 3.2 приведены значения precision, recall и F1-score по каждому классу. Наиболее высокое значение F1-score получено для класса male_14-17 — 0.8303. Также хорошие результаты показали классы female_14-17, male_45+, male_25-34 и female_25-34. Более сложными для модели оказались female_35-44 и female_18-24, что можно объяснить пересечением лексики между соседними возрастными группами.

Таблица 3.2 – Результаты классификации по классам

Класс	Precision	Recall	F1-score	Support
female_14-17	0.823	0.821	0.822	800
female_18-24	0.777	0.814	0.795	800
female_25-34	0.831	0.810	0.820	800
female_35-44	0.770	0.801	0.785	800
female_45+	0.836	0.794	0.814	800
male_14-17	0.832	0.829	0.830	800
male_18-24	0.807	0.816	0.812	800
male_25-34	0.837	0.806	0.821	800
male_35-44	0.793	0.794	0.793	800
male_45+	0.815	0.829	0.822	800
accuracy	–	–	0.811	8000
macro avg	0.812	0.811	0.812	8000

Примечание – Источник: собственная разработка

График функции потерь представлен на рисунке 3.1. На нем видно, что ошибка на обучающей выборке постепенно уменьшается, а ошибка на тестовой выборке после первых эпох начинает расти. Такая ситуация указывает на признаки переобучения: модель все лучше подстраивается под обучающие данные, но хуже обобщает часть новых текстов. При этом итоговая ассурасу остается выше 80%, поэтому модель можно считать работоспособной, но не идеально устойчивой.



Рисунок 3.1 – График изменения функции потерь при обучении

Примечание – Источник: собственная разработка

Матрица ошибок представлена на рисунке 3.2. Основная часть значений сосредоточена на главной диагонали, что означает правильную классификацию большинства объектов. Ошибки чаще возникают между соседними возрастными группами внутри одного пола, а также между подростковыми группами разных полов. Такая картина является ожидаемой, поскольку короткие Telegram-сообщения могут содержать общие разговорные признаки, не уникальные для конкретной группы.

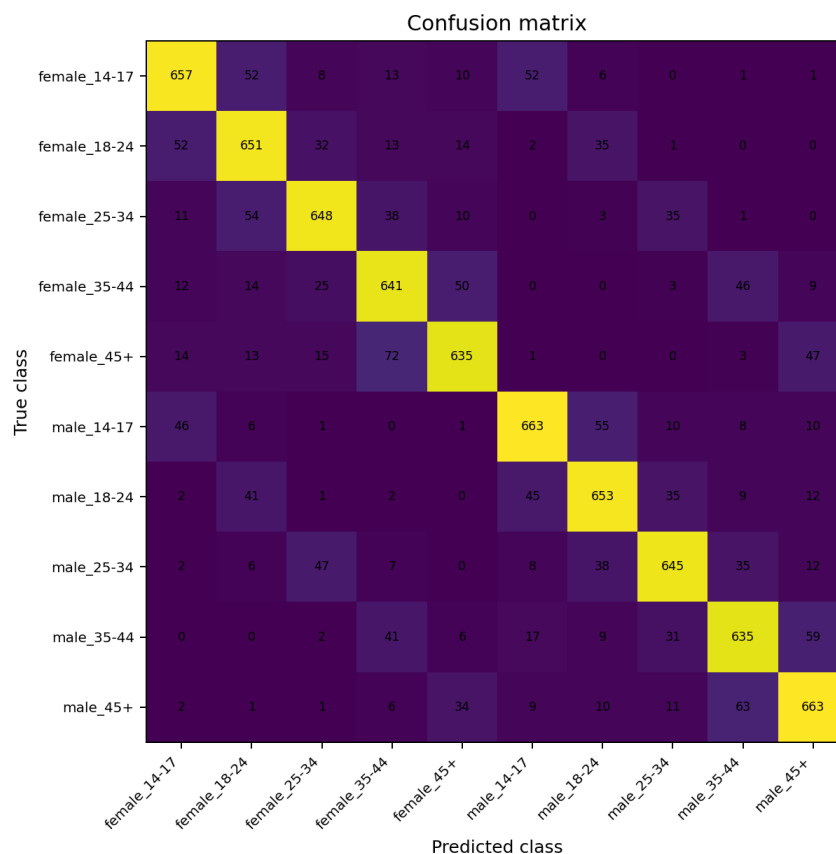


Рисунок 3.2 – Матрица ошибок модели
Примечание – Источник: собственная разработка

Например, для класса `female_14-17` модель правильно определила 657 объектов из 800. Часть ошибок пришлась на `female_18-24` и `male_14-17`. Это показывает, что часть текстовых признаков подростковых сообщений пересекается между полами и соседними возрастными категориями. Для класса `male_45+` модель правильно определила 663 объекта, но часть сообщений была отнесена к `male_35-44`, что также можно объяснить близостью соседних старших групп.

Если рассматривать результат в целом, модель достигла поставленной практической цели. Она способна по тексту предсказать объединенный возрастно-половой класс автора с точностью около 81%. При случайном выборе из десяти классов ожидаемая точность составляла бы около 10%, поэтому полученный результат показывает, что модель действительно извлекает полезные признаки из текстовых данных.

3.3 Анализ ошибок и направления улучшения

Главная причина ошибок связана с тем, что пол и возраст не всегда явно выражаются в одном сообщении. Пользователь может писать нейтрально,

обсуждать общие темы или использовать лексику, характерную для разных групп. Кроме того, в Telegram один и тот же человек может менять стиль в зависимости от ситуации: в канале писать более официально, а в комментариях — кратко и эмоционально.

Вторая причина — ограниченность TF-IDF-представления. TF-IDF хорошо отражает частотность слов и коротких словосочетаний, но плохо учитывает глубокий смысл сообщения и дальние зависимости между словами. Например, два сообщения могут быть похожи по теме, но различаться по стилистике, или наоборот — иметь разные темы, но похожий авторский стиль. Более сложные языковые модели могли бы учитывать контекст лучше.

Третья причина — признаки переобучения, заметные на графике функции потерь. Validation loss увеличивается, хотя train loss уменьшается. Это говорит о том, что модель частично запоминает обучающие данные. Для уменьшения переобучения можно использовать early stopping, уменьшить число эпох, усилить регуляризацию, изменить размер скрытых слоев или расширить и разнообразить набор данных.

Одним из улучшений может быть отдельное предсказание пола и возраста двумя выходными головами модели. В текущей работе используется один объединенный класс. Такой подход прост, но ошибка в возрасте и ошибка в поле считаются одинаково как неправильный класс. Если решать две задачи отдельно, можно увидеть, какая характеристика определяется лучше, и более гибко анализировать результаты.

Еще одним направлением является использование эмбедингов или трансформерных моделей для русского языка. Такие модели лучше учитывают контекст и могут давать более высокое качество, особенно на коротких и неоднозначных текстах. Однако они требуют больше вычислительных ресурсов и сложнее объясняются в рамках базовой курсовой работы. Поэтому для данной реализации выбран более простой и воспроизводимый подход TF-IDF + MLP.

Также возможно улучшить процедуру проверки модели. В данной работе используется одно разделение на обучающую и тестовую части. Для более надежной оценки можно применить кросс-валидацию или провести несколько запусков с разными значениями random_state. Это позволило бы оценить устойчивость результата и уменьшить влияние конкретного разбиения данных.

Несмотря на ограничения, разработанная модель показывает приемлемый результат для учебной задачи. Она демонстрирует весь цикл машинного обучения: подготовку данных, векторизацию, построение нейронной сети, обучение, расчет метрик и анализ ошибок. Полученная accuracy выше 80%

подтверждает, что выбранная схема обработки текста работает и может быть использована как базовый вариант для дальнейших исследований.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта была рассмотрена задача автоматизированного профилирования личности на основе текстового анализа данных Telegram. Основное внимание уделено предсказанию пола и возрастной группы автора текста по постам и комментариям. Задача была представлена как многоклассовая классификация, где каждый класс объединяет два признака: пол и возрастную группу.

В первой главе были рассмотрены теоретические основы авторского профилирования, особенности анализа пользовательских текстов в социальных медиа, способы представления текстовых данных и метрики качества классификации. Было показано, что задача профилирования автора имеет вероятностный характер и не должна рассматриваться как точное установление личности пользователя.

Во второй главе была спроектирована схема решения задачи. Подготовлен учебный набор данных объемом 40000 текстов, включающий десять сбалансированных классов. Для представления текста выбран метод TF-IDF с учетом униграмм и биграмм. В качестве модели использована полносвязная нейронная сеть MLP с тремя линейными слоями, функцией активации ReLU и dropout-регуляризацией.

В третьей главе была описана программная реализация модели на языке Python. Для работы использовались библиотеки pandas, NumPy, scikit-learn и PyTorch. Реализованы загрузка данных, стратифицированное разделение выборки, обучение TF-IDF-векторизатора, кодирование меток, обучение нейронной сети, сохранение модели и построение графиков результатов.

По результатам тестирования модель достигла общей точности 81.14%. Среднее значение F1-меры по всем классам составило 0.8115. Это показывает, что модель работает существенно лучше случайного выбора и извлекает полезные признаки из текстовых сообщений. При этом качество не является искусственно идеальным, что позволяет анализировать ошибки и ограничения модели.

Анализ матрицы ошибок показал, что чаще всего модель путает соседние возрастные группы и некоторые группы с близким стилем сообщений. Это объясняется тем, что речевые признаки пола и возраста могут пересекаться, особенно в коротких комментариях. График функции потерь показал признаки переобучения, поэтому в дальнейшей работе целесообразно использовать early stopping и дополнительные методы регуляризации.

К основным проблемам исследуемой задачи относятся неоднозначность коротких текстов, пересечение лексики между группами, зависимость стиля от темы сообщения и ограниченность TF-IDF-представления. Эти проблемы не позволяют получить абсолютно точную классификацию, но являются естественными для анализа пользовательского текста.

Пути дальнейшего совершенствования включают расширение набора данных, использование более сложных языковых моделей, отдельное предсказание пола и возраста, проведение кросс-валидации, подбор гиперпараметров и применение ранней остановки обучения. Также возможно добавление признаков длины сообщения, пунктуации, частоты местоимений и других стилистических характеристик.

Таким образом, цель курсового проекта достигнута. Была разработана и протестирована модель нейронной сети для предсказания пола и возраста автора Telegram-текста. Полученные результаты подтверждают применимость методов машинного обучения и нейронных сетей для учебной задачи авторского профилирования на текстовых данных.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Гудфеллоу, И. Глубокое обучение / И. Гудфеллоу, Й. Бенджио, А. Курвилль; пер. с англ. – Москва: ДМК Пресс, 2018. – 652 с.
2. Bishop, C. M. Pattern Recognition and Machine Learning / C. M. Bishop. – New York: Springer, 2006. – 738 p.
3. Géron, A. Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow / A. Géron. – 3rd ed. – Sebastopol: O'Reilly Media, 2022. – 861 p.
4. Raschka, S. Machine Learning with PyTorch and Scikit-Learn / S. Raschka, Y. Liu, V. Mirjalili. – Birmingham: Packt Publishing, 2022. – 774 p.
5. Chollet, F. Deep Learning with Python / F. Chollet. – 2nd ed. – Shelter Island: Manning Publications, 2021. – 504 p.
6. Jurafsky, D. Speech and Language Processing [Электронный ресурс] / D. Jurafsky, J. H. Martin. – Режим доступа: <https://web.stanford.edu/~jurafsky/slp3/>. – Дата доступа: 19.05.2026.
7. Manning, C. D. Introduction to Information Retrieval / C. D. Manning, P. Raghavan, H. Schütze. – Cambridge: Cambridge University Press, 2008. – 482 p.
8. Sebastiani, F. Machine learning in automated text categorization / F. Sebastiani // ACM Computing Surveys. – 2002. – Vol. 34, № 1. – P. 1–47.
9. Joachims, T. Text categorization with Support Vector Machines: learning with many relevant features / T. Joachims // Proceedings of ECML. – 1998. – P. 137–142.
10. Koppel, M. Automatically categorizing written texts by author gender / M. Koppel, S. Argamon, A. R. Shimoni // Literary and Linguistic Computing. – 2002. – Vol. 17, № 4. – P. 401–412.
11. Argamon, S. Mining the blogosphere: age, gender and the varieties of self-expression / S. Argamon [et al.] // First Monday. – 2007. – Vol. 12, № 9.
12. Schwartz, H. A. Personality, gender, and age in the language of social media: the open-vocabulary approach / H. A. Schwartz [et al.] // PLOS ONE. – 2013. – Vol. 8, № 9.
13. Rangel, F. Overview of the 3rd author profiling task at PAN 2015 / F. Rangel [et al.] // Working Notes of CLEF. – 2015.
14. Nguyen, D. Author age prediction from text using linear regression / D. Nguyen, A. Smith, C. Rosé // Proceedings of LaTeCH. – 2011. – P. 115–123.
15. Pennebaker, J. W. The Secret Life of Pronouns / J. W. Pennebaker. – New York: Bloomsbury Press, 2011. – 368 p.
16. Pedregosa, F. Scikit-learn: machine learning in Python / F. Pedregosa [et al.] // Journal of Machine Learning Research. – 2011. – Vol. 12. – P. 2825–2830.

17. Paszke, A. PyTorch: an imperative style, high-performance deep learning library / A. Paszke [et al.] // Advances in Neural Information Processing Systems. – 2019. – Vol. 32.
18. Kingma, D. P. Adam: a method for stochastic optimization / D. P. Kingma, J. Ba // International Conference on Learning Representations. – 2015.
19. Hinton, G. Improving neural networks by preventing co-adaptation of feature detectors / G. Hinton [et al.] // arXiv preprint arXiv:1207.0580. – 2012.
20. Srivastava, N. Dropout: a simple way to prevent neural networks from overfitting / N. Srivastava [et al.] // Journal of Machine Learning Research. – 2014. – Vol. 15. – P. 1929–1958.
21. CrossEntropyLoss [Электронный ресурс] // PyTorch Documentation. – Режим доступа: <https://docs.pytorch.org/docs/stable/generated/torch.nn.modules.loss.CrossEntropyLoss.html>. – Дата доступа: 19.05.2026.
22. Adam [Электронный ресурс] // PyTorch Documentation. – Режим доступа: <https://docs.pytorch.org/docs/stable/generated/torch.optim.Adam.html>. – Дата доступа: 19.05.2026.
23. torch.utils.data [Электронный ресурс] // PyTorch Documentation. – Режим доступа: <https://docs.pytorch.org/docs/stable/data.html>. – Дата доступа: 19.05.2026.
24. TfidfVectorizer [Электронный ресурс] // scikit-learn Documentation. – Режим доступа: https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html. – Дата доступа: 19.05.2026.
25. train_test_split [Электронный ресурс] // scikit-learn Documentation. – Режим доступа: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html. – Дата доступа: 19.05.2026.
26. classification_report [Электронный ресурс] // scikit-learn Documentation. – Режим доступа: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.classification_report.html. – Дата доступа: 19.05.2026.
27. confusion_matrix [Электронный ресурс] // scikit-learn Documentation. – Режим доступа: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.confusion_matrix.html. – Дата доступа: 19.05.2026.

28. `pandas.read_csv` [Электронный ресурс] // pandas Documentation. – Режим доступа: https://pandas.pydata.org/docs/reference/api/pandas.read_csv.html. – Дата доступа: 19.05.2026.

29. `numpy.ndarray` [Электронный ресурс] // NumPy Documentation. – Режим доступа: <https://numpy.org/doc/stable/reference/generated/numpy.ndarray.html>. – Дата доступа: 19.05.2026.

30. McKinney, W. Data structures for statistical computing in Python / W. McKinney // Proceedings of the 9th Python in Science Conference. – 2010. – P. 56–61.

31. Python 3.12 documentation [Электронный ресурс] // Python Software Foundation. – Режим доступа: <https://docs.python.org/3/>. – Дата доступа: 19.05.2026.

ПРИЛОЖЕНИЕ А

Фрагменты программной реализации

В приложении приведены основные фрагменты программной реализации, необходимые для понимания структуры модели и процесса обучения. Полные служебные фрагменты, связанные с построением графиков и сохранением файлов, в приложение не включаются, чтобы не перегружать отчет.

```
class TextProfileMLP(nn.Module):
    def __init__(self, input_dim: int, num_classes: int) -> None:
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, 256),
            nn.ReLU(),
            nn.Dropout(0.35),
            nn.Linear(256, 128),
            nn.ReLU(),
            nn.Dropout(0.25),
            nn.Linear(128, num_classes),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.net(x)
```

Листинг А.1 – Архитектура нейронной сети

Примечание – Источник: собственная разработка

```
vectorizer = TfidfVectorizer(
    lowercase=True,
    ngram_range=(1, 2),
    max_features=args.max_features,
    min_df=2,
)
x_train = vectorizer.fit_transform(x_train_text)
x_test = vectorizer.transform(x_test_text)
```

Листинг А.2 – Векторизация текста методом TF-IDF

Примечание – Источник: собственная разработка

```
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

for epoch in range(args.epochs):
    model.train()
    for xb, yb in train_loader:
        optimizer.zero_grad()
        logits = model(xb)
        loss = criterion(logits, yb)
        loss.backward()
        optimizer.step()
```

Листинг А.3 – Основной цикл обучения модели

Примечание – Источник: собственная разработка

```
model.eval()
with torch.no_grad():
    logits = model(x_test_tensor)
    y_pred = logits.argmax(dim=1).cpu().numpy()

report = classification_report(
    y_test,
```

```
y_pred,  
target_names=labels,  
output_dict=True,  
zero_division=0,  
)
```

Листинг А.4 – Получение предсказаний и отчета качества

Примечание – Источник: собственная разработка

ПРИЛОЖЕНИЕ Б

Отчет качества модели

В приложении приведена сводная таблица итоговых средних метрик, полученных на тестовой выборке. Тестовая выборка содержит 8000 текстов, по 800 текстов на каждый класс.

Таблица Б.1 – Сводные значения качества модели

Показатель	Precision	Recall	F1-score	Support
accuracy	–	–	0.811375	8000
macro avg	0.811983	0.811375	0.811501	8000
weighted avg	0.811983	0.811375	0.811501	8000

Примечание – Источник: собственная разработка

Полученный результат показывает, что модель правильно классифицирует немного больше восьми из десяти текстов тестовой выборки. Для учебной задачи это является приемлемым результатом, так как модель решает не бинарную, а десятиклассовую задачу, где случайный выбор давал бы около 10% точности.

ПРИЛОЖЕНИЕ В

Порядок запуска эксперимента

Данное приложение содержит краткую инструкцию по запуску эксперимента в среде Python. Инструкция нужна для того, чтобы результаты работы можно было повторить на другом компьютере без изменения программного кода. В качестве рабочей среды может использоваться PyCharm, Visual Studio Code или обычный терминал Windows PowerShell.

Перед запуском необходимо открыть корневую папку проекта, в которой находятся каталоги data, src и файл requirements.txt. Виртуальное окружение рекомендуется создавать внутри проекта, чтобы зависимости курсовой работы не смешивались с глобальными пакетами Python. Такой подход удобен при переносе проекта на другой компьютер и снижает вероятность конфликтов версий библиотек.

После создания виртуального окружения устанавливаются зависимости из файла requirements.txt. Основными библиотеками являются pandas, numpy, scikit-learn, torch, matplotlib и joblib. Эти библиотеки обеспечивают загрузку данных, числовые вычисления, векторизацию текста, обучение нейронной сети, построение графиков и сохранение объектов модели.

Далее запускается основной скрипт train_model.py. В параметре --data указывается путь к CSV-файлу с набором данных, в параметре --out-dir — папка, куда будут сохранены результаты. Параметр --epochs задает число эпох обучения, --batch-size — размер батча, а --max-features — максимальное количество TF-IDF-признаков.

После завершения обучения в консоли выводится отчет classification_report. Он содержит precision, recall, F1-score и support для каждого класса. Кроме того, эти же данные сохраняются в файл classification_report.json, что позволяет использовать их в отчете без ручного переписывания значений из консоли.

Если модель нужно проверить на отдельной фразе, используется скрипт predict_author_profile.py. Он загружает сохраненные веса модели, векторизатор и кодировщик меток, затем преобразует новый текст в TF-IDF-вектор и выводит предсказанный класс. Важно, что для предсказания используется тот же векторизатор, который был обучен во время обучения модели.

При повторном запуске эксперимента рекомендуется сохранять результаты в новую папку, например outputs_diverse_40000_v2. Это позволяет сравнивать разные варианты обучения и не перезаписывать предыдущие

артефакты. Старые папки можно удалить только после того, как результаты были перенесены в отчет или сохранены отдельно.

Если во время установки библиотек возникают ошибки, в первую очередь следует проверить выбранный интерпретатор Python и дождаться завершения индексации виртуального окружения в PyCharm. Также полезно обновить pip командой `python -m pip install --upgrade pip`. В большинстве случаев после корректного выбора интерпретатора зависимости устанавливаются без дополнительной настройки.

Таблица В.1 – Основные команды для запуска проекта

Этап	Команда
Создание окружения	<code>python -m venv .venv</code>
Активация окружения Windows	<code>.venv\Scripts\activate</code>
Установка зависимостей	<code>pip install -r requirements.txt</code>
Запуск обучения	<code>python src/train_model.py --data data/telegram_author_profile_synthetic_diverse_40000.csv --out-dir outputs_diverse_40000 --epochs 25 --batch-size 128 --max-features 3000</code>
Проверка отдельного текста	<code>python src/predict_author_profile.py --model-dir outputs_diverse_40000 -- text "текст сообщения"</code>

Примечание – Источник: собственная разработка

```
# пример полного запуска в PowerShell
cd C:\Users\chaik\Documents\coursework_author_profiling
.venv\Scripts\activate
pip install -r requirements.txt
python src/train_model.py --data
data/telegram_author_profile_synthetic_diverse_40000.csv --out-dir
outputs_diverse_40000 --epochs 25 --batch-size 128 --max-features 3000
```

Листинг В.1 – Пример запуска обучения в PowerShell

Примечание – Источник: собственная разработка

ПРИЛОЖЕНИЕ Г

Описание выходных файлов эксперимента

В результате обучения модель создает несколько выходных файлов. Они нужны не только для демонстрации результата, но и для повторного использования модели. Если сохранить только изображение графика или отчет в консоли, эксперимент будет трудно воспроизвести. Поэтому программа сохраняет как визуальные материалы, так и служебные файлы, необходимые для последующего предсказания.

Файл `author_profile_mlp.pt` содержит веса нейронной сети. Он не хранит саму архитектуру в виде полного объекта, поэтому при загрузке модели необходимо создать такой же класс `TextProfileMLP`, а затем загрузить веса с помощью `load_state_dict`. Это стандартный и достаточно безопасный способ сохранения моделей PyTorch для учебных проектов.

Файл `tfidf_vectorizer.joblib` содержит обученный TF-IDF-векторизатор. Его сохранение обязательно, потому что порядок признаков и словарь должны совпадать с теми, которые использовались при обучении. Если заново обучить векторизатор на другом наборе текстов, входные признаки изменятся, и веса модели уже не будут соответствовать новым векторам.

Файл `label_encoder.joblib` хранит соответствие между строковыми названиями классов и их числовыми индексами. Без него модель может выдать номер класса, но будет неудобно понять, какой именно возрастно-половой группе он соответствует. Поэтому кодировщик меток сохраняется вместе с моделью и векторизатором.

Файлы `classification_report.txt` и `classification_report.json` содержат итоговые метрики. Текстовый файл удобен для быстрого просмотра человеком, а JSON-файл удобен для программной обработки. В данной работе значения из JSON-файла использованы для формирования таблицы результатов в третьей главе.

Файл `confusion_matrix.csv` содержит матрицу ошибок в табличном виде. На его основе можно построить график или дополнительно проанализировать, какие классы чаще всего путаются между собой. Изображение `confusion_matrix.png` используется в тексте отчета как визуальное подтверждение качества модели.

Файл `training_history.csv` хранит историю обучения по эпохам. В нем находятся значения `train loss`, `validation loss` и `validation accuracy`. Из этого файла строится график `training_curve.png`. Такой график помогает понять, не происходит ли переобучение и насколько стабильно модель обучается.

Таким образом, выходные файлы эксперимента образуют полный комплект артефактов: модель, средства предобработки, числовые метрики и визуализации. Это делает работу воспроизводимой и позволяет при необходимости повторить эксперимент, изменить параметры или сравнить несколько вариантов модели.

Таблица Г.1 – Назначение выходных файлов

Файл	Назначение
author_profile_mlp.pt	Веса обученной нейронной сети
tfidf_vectorizer.joblib	Словарь и параметры TF-IDF-векторизатора
label_encoder.joblib	Кодировщик строковых меток классов
classification_report.txt	Текстовая версия отчета качества
classification_report.json	Структурированный отчет качества для программной обработки
confusion_matrix.csv	Матрица ошибок в табличном виде
confusion_matrix.png	Изображение матрицы ошибок
training_history.csv	История значений функции потерь и точности
training_curve.png	График обучения и валидации

Примечание – Источник: собственная разработка

При защите работы удобно объяснять, что эти файлы относятся к разным этапам эксперимента. Файлы модели и предобработки нужны для повторного применения решения, а файлы с метриками и графиками нужны для подтверждения качества. Такое разделение показывает, что эксперимент не ограничился единичным запуском программы, а был оформлен как воспроизводимый процесс.

Если после обучения требуется проверить модель на новом сообщении, программа сначала загружает `tfidf_vectorizer.joblib` и преобразует текст в такой же набор признаков, как и при обучении. Затем создается экземпляр нейронной сети с тем же количеством входных признаков и классов, после чего загружается файл `author_profile_mlp.pt`. Только после этого можно корректно получить предсказание.

Если заменить векторизатор или изменить параметр `max_features` без повторного обучения модели, результат станет некорректным. Причина в том, что веса первого слоя нейронной сети соответствуют конкретному порядку TF-IDF-признаков. Поэтому в практических проектах модель, векторизатор и кодировщик меток следует хранить как единый комплект.